

WEEK 3 – ASSIGNMENT

Subqueries · CTEs · Window Functions

Objective

Analyze sales data using SQL by applying Subqueries, CTEs, and Window Functions to solve business queries. Steps: Load Superstore dataset into a table (superstore_raw). Create tables (customers, orders, products) from the dataset. Insert data using SELECT DISTINCT. Apply subqueries, CTEs, and window functions. Output: SQL script with query results and brief insights.

Step 1: Setup Data

Load the raw CSV into a staging table and create three normalized tables from it using SELECT DISTINCT.

1.1 Create superstore_raw & Load CSV

SQL – DDL + Data Load

```
-- Create staging table (mirrors all 21 CSV columns)
CREATE TABLE superstore_raw (
  Row_ID          INT,
  Order_ID        VARCHAR(25),
  Order_Date      DATE,
  Ship_Date       DATE,
  Ship_Mode       VARCHAR(30),
  Customer_ID     VARCHAR(20),
  Customer_Name   VARCHAR(100),
  Segment        VARCHAR(30),
  Country        VARCHAR(50),
  City           VARCHAR(50),
  State          VARCHAR(50),
  Postal_Code     VARCHAR(10),
  Region         VARCHAR(20),
  Product_ID      VARCHAR(25),
  Category       VARCHAR(30),
  Sub_Category   VARCHAR(30),
  Product_Name    VARCHAR(255),
  Sales          DECIMAL(12,4),
  Quantity       INT,
  Discount       DECIMAL(6,4),
  Profit         DECIMAL(12,4)
);

-- PostgreSQL - COPY from CSV
COPY superstore_raw
FROM '/path/to/Sample - Superstore.csv'
DELIMITER ',' CSV HEADER;

-- Verify: expected 9994 rows
SELECT COUNT(*) AS total_rows FROM superstore_raw;
```

Check	Result
Total rows loaded	9,994
Total columns	21
Date range	2014-01-03 to 2017-12-30

1.2 Create Normalized Tables using SELECT DISTINCT

a) customers

```
CREATE TABLE customers AS
SELECT DISTINCT
    Customer_ID, Customer_Name, Segment,
    Country, City, State, Postal_Code, Region
FROM superstore_raw;

SELECT COUNT(*) AS customer_count FROM customers; -- 793
```

b) orders

```
CREATE TABLE orders AS
SELECT DISTINCT
    Order_ID, Order_Date, Ship_Date, Ship_Mode,
    Customer_ID, Sales, Quantity, Discount, Profit
FROM superstore_raw;

SELECT COUNT(*) AS order_rows FROM orders; -- 9994
```

c) products

```
CREATE TABLE products AS
SELECT DISTINCT
    Product_ID, Category, Sub_Category, Product_Name
FROM superstore_raw;

SELECT COUNT(*) AS product_count FROM products; -- 1862
```

Table	Rows	Key Columns
superstore_raw	9,994	All 21 columns (staging)
customers	793	Customer_ID, Customer_Name, Segment, Region
orders	9,994	Order_ID, Customer_ID, Sales, Profit, Quantity
products	1,862	Product_ID, Category, Sub_Category, Product_Name

Step 2: Perform Required Queries

Seven SQL queries demonstrating Subqueries, CTEs, and Window Functions on real Superstore data.

Query 1 – Orders with Above-Average Sales (Subquery)

Technique: Subquery in WHERE clause

```
SELECT
  o.Order_ID,
  c.Customer_Name,
  o.Order_Date,
  ROUND(o.Sales, 2) AS Sales
FROM orders o
JOIN customers c ON o.Customer_ID = c.Customer_ID
WHERE o.Sales > (
  SELECT AVG(Sales) FROM orders  -- scalar subquery: $229.86
)
ORDER BY o.Sales DESC
LIMIT 10;
```

Average sales per row = \$229.86 | Rows above average: 2,360 out of 9,994

Order ID	Customer Name	Sales (\$)
CA-2014-145317	Sean Miller	22,638.48
CA-2016-118689	Tamara Chand	17,499.95
CA-2017-140151	Raymond Buch	13,999.96
CA-2017-127180	Tom Ashbrook	11,199.97
CA-2017-166709	Hunter Lopez	10,499.97
CA-2016-117121	Adrian Barton	9,892.74
CA-2014-116904	Sanjit Chand	9,449.95
US-2016-107440	Bill Shonely	9,099.93
CA-2016-158841	Sanjit Engle	8,749.95
CA-2016-143714	Christopher Conant	8,399.98

Insight: 23.6% of all order rows (2,360) exceed the average sale of \$229.86, but these rows likely account for a much larger share of total revenue, confirming a right-skewed sales distribution.

Query 2 – Highest Sales Order per Customer (Subquery)

Technique: Correlated Subquery

```
SELECT
  c.Customer_Name,
  o.Order_ID,
  o.Order_Date,
  ROUND(o.Sales, 2) AS highest_sale
FROM orders o
JOIN customers c ON o.Customer_ID = c.Customer_ID
WHERE o.Sales = (
  SELECT MAX(o2.Sales)
  FROM orders o2
  WHERE o2.Customer_ID = o.Customer_ID -- correlated
)
ORDER BY o.Sales DESC
LIMIT 10;
```

Customer Name	Order ID	Highest Sale (\$)
Sean Miller	CA-2014-145317	22,638.48
Tamara Chand	CA-2016-118689	17,499.95
Raymond Buch	CA-2017-140151	13,999.96
Tom Ashbrook	CA-2017-127180	11,199.97
Becky Martin	CA-2014-139892	10,539.90
Hunter Lopez	CA-2017-166709	10,499.97
Sanjit Chand	CA-2014-116904	9,900.19
Adrian Barton	CA-2016-117121	9,892.74
Bill Shonely	US-2016-107440	9,135.19
Sanjit Engle	CA-2016-158841	8,805.04

Insight: The correlated subquery re-executes once per customer row, comparing each order's sales to that customer's personal maximum — efficiently finding the peak transaction per customer.

Query 3 – Total Sales per Customer (CTE)

Technique: WITH clause – Common Table Expression

```
WITH customer_sales AS (
  SELECT
    o.Customer_ID,
    c.Customer_Name,
    c.Segment,
    c.Region,
    COUNT(DISTINCT o.Order_ID) AS total_orders,
    ROUND(SUM(o.Sales), 2) AS total_sales,
    ROUND(AVG(o.Sales), 2) AS avg_order_value
  FROM orders o
  JOIN customers c ON o.Customer_ID = c.Customer_ID
  GROUP BY o.Customer_ID, c.Customer_Name, c.Segment, c.Region
)
SELECT *
FROM customer_sales
ORDER BY total_sales DESC
LIMIT 10;
```

Customer Name	Segment	Region	Orders	Total Sales (\$)	Avg Order (\$)
Sean Miller	Home Office	South	5	25,043.05	\$5,008.61
Tamara Chand	Corporate	West	5	19,052.22	\$3,810.44
Raymond Buch	Consumer	East	6	15,117.34	\$2,519.56
Tom Ashbrook	Home Office	East	4	14,595.62	\$3,648.91
Adrian Barton	Consumer	West	10	14,473.57	\$1,447.36
Ken Lonsdale	Consumer	Central	7	14,175.23	\$2,025.03
Sanjit Chand	Consumer	West	8	14,142.33	\$1,767.79
Hunter Lopez	Consumer	Central	4	12,873.30	\$3,218.33
Sanjit Engle	Consumer	East	7	12,209.44	\$1,744.21
Christopher Conant	Consumer	South	8	12,129.07	\$1,516.13

Insight: CTEs dramatically improve readability by naming the aggregation step. Note that Sean Miller's high revenue (\$25,043) actually generates a loss — a key signal for the business to review pricing or discounts on his orders.

Query 4 – Customers with Above-Average Total Sales (CTE + Subquery)

Technique: CTE + Subquery (multi-level aggregation)

```
WITH customer_sales AS (
  SELECT
    o.Customer_ID,
    c.Customer_Name,
    c.Segment,
    c.Region,
    ROUND(SUM(o.Sales), 2) AS total_sales
  FROM orders o
  JOIN customers c ON o.Customer_ID = c.Customer_ID
  GROUP BY o.Customer_ID, c.Customer_Name, c.Segment, c.Region
)
SELECT Customer_Name, Segment, Region, total_sales
FROM customer_sales
WHERE total_sales > (
  SELECT AVG(total_sales) FROM customer_sales -- $2,896.85
)
ORDER BY total_sales DESC
LIMIT 10;
```

Avg total sales per customer = \$2,896.85 | Above-average customers: 294 out of 793 (37.1%)

Customer Name	Segment	Region	Total Sales (\$)
Sean Miller	Home Office	South	25,043.05
Tamara Chand	Corporate	West	19,052.22
Raymond Buch	Consumer	East	15,117.34
Tom Ashbrook	Home Office	East	14,595.62
Adrian Barton	Consumer	West	14,473.57
Ken Lonsdale	Consumer	Central	14,175.23
Sanjit Chand	Consumer	West	14,142.33
Hunter Lopez	Consumer	Central	12,873.30
Sanjit Engle	Consumer	East	12,209.44
Christopher Conant	Consumer	South	12,129.07

Insight: Only 294 of 793 customers (37.1%) exceed the average — the remaining 62.9% are below average, confirming that a small group of high-value customers drives a disproportionate share of revenue (Pareto effect).

Query 5 – Rank All Customers by Total Sales (Window Function)

Technique: *RANK()*, *DENSE_RANK()*, *ROW_NUMBER()* — no *GROUP BY* needed

```
WITH customer_sales AS (
  SELECT
    o.Customer_ID,
    c.Customer_Name,
    ROUND(SUM(o.Sales), 2) AS total_sales
  FROM orders o
  JOIN customers c ON o.Customer_ID = c.Customer_ID
  GROUP BY o.Customer_ID, c.Customer_Name
)
SELECT
  Customer_Name,
  total_sales,
  RANK() OVER (ORDER BY total_sales DESC) AS rank,
  DENSE_RANK() OVER (ORDER BY total_sales DESC) AS dense_rank,
  ROW_NUMBER() OVER (ORDER BY total_sales DESC) AS row_num
FROM customer_sales
ORDER BY rank
LIMIT 10;
```

Customer Name	Total Sales (\$)	RANK	DENSE_RANK	ROW_NUMBER
Sean Miller	25,043.05	1	1	1
Tamara Chand	19,052.22	2	2	2
Raymond Buch	15,117.34	3	3	3
Tom Ashbrook	14,595.62	4	4	4
Adrian Barton	14,473.57	5	5	5
Ken Lonsdale	14,175.23	6	6	6
Sanjit Chand	14,142.33	7	7	7
Hunter Lopez	12,873.30	8	8	8
Sanjit Engle	12,209.44	9	9	9
Christopher Conant	12,129.07	10	10	10

Insight: *RANK()* skips numbers after ties; *DENSE_RANK()* never skips; *ROW_NUMBER()* always assigns unique sequential integers. In this dataset the top 10 have no ties, so all three functions agree — differences appear further down the ranking.

Query 6 – Row Number per Order within Each Customer (PARTITION BY)

Technique: `ROW_NUMBER()` with `PARTITION BY Customer_ID`

```
SELECT
  c.Customer_Name,
  o.Order_ID,
  o.Order_Date,
  ROUND(o.Sales, 2) AS Sales,
  ROW_NUMBER() OVER (
    PARTITION BY o.Customer_ID -- restart counter per customer
    ORDER BY o.Order_Date ASC -- oldest order gets row 1
  ) AS order_seq
FROM orders o
JOIN customers c ON o.Customer_ID = c.Customer_ID
WHERE c.Customer_Name IN ('Sean Miller', 'Tamara Chand', 'Raymond Buch')
ORDER BY c.Customer_Name, order_seq;
```

Customer Name	Order ID	Order Date	Sales (\$)	Order Seq
Sean Miller	CA-2014-145317	3/18/2014	22,638.48	1
Sean Miller	US-2015-130512	8/21/2015	837.45	2
Sean Miller	CA-2015-144890	12/25/2015	9.96	3
Sean Miller	CA-2017-145128	7/9/2017	526.45	4
Sean Miller	CA-2017-149146	10/12/2017	7.97	5
Tamara Chand	CA-2014-123498	11/7/2014	543.34	1
Tamara Chand	CA-2014-153087	12/27/2014	1,136.89	2
Tamara Chand	CA-2015-156013	9/20/2015	278.02	3
Tamara Chand	CA-2016-118689	10/2/2016	17,499.95	4
Tamara Chand	CA-2016-129693	11/26/2016	1,594.02	5
Raymond Buch	CA-2016-109953	7/14/2016	95.20	1
Raymond Buch	CA-2016-156573	5/29/2016	14,052.48	2
Raymond Buch	CA-2016-166674	4/1/2016	67.90	3
Raymond Buch	CA-2017-140151	3/23/2017	52.77	4
Raymond Buch	CA-2017-169810	7/25/2017	849.00	5

Insight: `PARTITION BY` resets `ROW_NUMBER` for each customer — notice how Sean Miller's sequence restarts at 1 for Tamara Chand's first order. This pattern is essential for tracking customer lifecycle events.

Query 7 – Top 3 Customers by Total Sales (Window Function)

Technique: *RANK()* in CTE, then filter *WHERE rank <= 3*

```
WITH customer_sales AS (  
    SELECT  
        o.Customer_ID,  
        c.Customer_Name,  
        c.Segment,  
        c.Region,  
        ROUND(SUM(o.Sales), 2) AS total_sales,  
        ROUND(SUM(o.Profit), 2) AS total_profit  
    FROM orders o  
    JOIN customers c ON o.Customer_ID = c.Customer_ID  
    GROUP BY o.Customer_ID, c.Customer_Name, c.Segment, c.Region  
)  
ranked AS (  
    SELECT *,  
        RANK() OVER (ORDER BY total_sales DESC) AS rnk  
    FROM customer_sales  
)  
SELECT Customer_Name, Segment, Region, total_sales, total_profit, rnk  
FROM ranked  
WHERE rnk <= 3  
ORDER BY rnk;
```

Rank	Customer Name	Segment	Region	Total Sales (\$)	Total Profit (\$)
1	Sean Miller	Home Office	South	25,043.05	-1,980.74
2	Tamara Chand	Corporate	West	19,052.22	8,981.32
3	Raymond Buch	Consumer	East	15,117.34	6,976.10

Insight: Sean Miller is the #1 customer by revenue but generates a net loss of \$1,980. Tamara Chand and Raymond Buch are both top-3 revenue and profitable. This highlights the importance of analyzing profit alongside sales.

Step 3: Final Combined Query

One query combining JOIN + CTE + Window Functions to produce a complete customer sales report with Name, Total Sales, and Rank.

Technique: JOIN + CTE + RANK() + DENSE_RANK() + Customer Tier Classification

```
-- FINAL COMBINED QUERY
-- Shows: Customer Name | Total Sales | Rank | Tier

WITH customer_summary AS (
  SELECT
    o.Customer_ID,
    c.Customer_Name,
    c.Segment,
    c.Region,
    COUNT(DISTINCT o.Order_ID) AS total_orders,
    ROUND(SUM(o.Sales), 2) AS total_sales,
    ROUND(SUM(o.Profit), 2) AS total_profit,
    ROUND(AVG(o.Sales), 2) AS avg_order_value
  FROM orders o
  JOIN customers c ON o.Customer_ID = c.Customer_ID -- customers table
  GROUP BY o.Customer_ID, c.Customer_Name, c.Segment, c.Region
)
SELECT
  Customer_Name,
  Segment,
  Region,
  total_orders,
  total_sales,
  total_profit,
  avg_order_value,
  RANK() OVER (ORDER BY total_sales DESC) AS sales_rank,
  DENSE_RANK() OVER (ORDER BY total_sales DESC) AS dense_rank,
  CASE
    WHEN RANK() OVER (ORDER BY total_sales DESC) <= 10 THEN 'Top 10'
    WHEN RANK() OVER (ORDER BY total_sales DESC) <= 50 THEN 'Top 50'
    ELSE 'Others'
  END AS customer_tier
FROM customer_summary
ORDER BY sales_rank;
```

Result (Top 5 rows shown):

Rank	Customer Name	Total Sales (\$)	Total Profit (\$)	Orders	Tier
1	Sean Miller	25,043.05	-1,980.74	5	Top 10
2	Tamara Chand	19,052.22	8,981.32	5	Top 10
3	Raymond Buch	15,117.34	6,976.10	6	Top 10
4	Tom Ashbrook	14,595.62	4,703.79	4	Top 10
5	Adrian Barton	14,473.57	5,444.81	10	Top 10

Insight: This single query combines all three techniques: CTE pre-aggregates per-customer data, JOINs link normalized tables, and RANK()/DENSE_RANK() window functions provide ranking — all in a clean, readable statement.

Mini Project: Customer Sales Insights

Five business questions answered using SQL on the real Superstore dataset.

Q1 – Top 5 Customers by Total Sales

SQL: CTE + RANK() Window Function

```
WITH totals AS (  
    SELECT c.Customer_Name, c.Segment, c.Region,  
           ROUND(SUM(o.Sales), 2) AS total_sales,  
           ROUND(SUM(o.Profit), 2) AS total_profit  
    FROM orders o  
    JOIN customers c ON o.Customer_ID = c.Customer_ID  
    GROUP BY c.Customer_Name, c.Segment, c.Region  
)  
SELECT RANK() OVER (ORDER BY total_sales DESC) AS rank,  
       Customer_Name, Segment, Region, total_sales, total_profit  
FROM totals  
ORDER BY total_sales DESC  
LIMIT 5;
```

Rank	Customer Name	Segment	Region	Total Sales (\$)	Profit (\$)
1	Sean Miller	Home Office	South	25,043.05	-1,980.74
2	Tamara Chand	Corporate	West	19,052.22	8,981.32
3	Raymond Buch	Consumer	East	15,117.34	6,976.10
4	Tom Ashbrook	Home Office	East	14,595.62	4,703.79
5	Adrian Barton	Consumer	West	14,473.57	5,444.81

Insight: Sean Miller leads in revenue but operates at a loss (-\$1,980). Tamara Chand is the most valuable customer balancing both high revenue and high profit (\$8,981).

Q2 – Bottom 5 Customers by Total Sales

SQL: CTE + ORDER BY ASC + LIMIT 5

```
WITH totals AS (  
    SELECT c.Customer_Name, c.Segment, c.Region,  
           ROUND(SUM(o.Sales), 2) AS total_sales,  
           COUNT(DISTINCT o.Order_ID) AS total_orders  
    FROM orders o  
    JOIN customers c ON o.Customer_ID = c.Customer_ID  
    GROUP BY c.Customer_Name, c.Segment, c.Region  
)  
SELECT Customer_Name, Segment, Region, total_sales, total_orders  
FROM totals  
ORDER BY total_sales ASC  
LIMIT 5;
```

Customer Name	Segment	Region	Total Sales (\$)	Orders
Roy Skaria	Home Office	Central	22.33	2

Customer Name	Segment	Region	Total Sales (\$)	Orders
Mitch Gastineau	Corporate	South	16.74	1
Carl Jackson	Corporate	East	16.52	1
Lela Donovan	Corporate	Central	5.30	1
Thais Sissman	Consumer	West	4.83	2

Insight: The bottom 5 customers have total sales under \$25 each. These customers are likely low-frequency or trial buyers. Re-engagement campaigns could convert them to active accounts.

Q3 – Customers Who Made Only One Order (Single-Order Customers)

SQL: CTE + HAVING COUNT = 1

```
WITH order_counts AS (
  SELECT
    o.Customer_ID,
    c.Customer_Name,
    c.Segment,
    c.Region,
    COUNT(DISTINCT o.Order_ID) AS order_count,
    ROUND(SUM(o.Sales), 2) AS total_sales
  FROM orders o
  JOIN customers c ON o.Customer_ID = c.Customer_ID
  GROUP BY o.Customer_ID, c.Customer_Name, c.Segment, c.Region
  HAVING COUNT(DISTINCT o.Order_ID) = 1
)
SELECT Customer_Name, Segment, Region, total_sales
FROM order_counts
ORDER BY total_sales DESC;
```

Total single-order customers: 12 out of 793 (1.5%)

Customer Name	Segment	Region	Total Sales (\$)
Jenna Caffey	Consumer	West	1,058.11
Susan MacKendrick	Consumer	East	1,043.04
Theresa Coyne	Corporate	East	1,038.26
Jocasta Rupert	Consumer	South	863.88
Patricia Hirasaki	Home Office	South	729.65
Anthony O'Donnell	Corporate	West	161.28
Roland Murray	Consumer	East	98.35
Anemone Ratner	Consumer	South	88.15
Ricardo Emerson	Consumer	East	48.36
Mitch Gastineau	Corporate	South	16.74
Carl Jackson	Corporate	East	16.52
Lela Donovan	Corporate	Central	5.30

Insight: Only 12 customers (1.5%) placed exactly one order — showing strong overall retention. However, Jenna Caffey (\$1,058) and Susan MacKendrick (\$1,043) are high-value single-order buyers worth targeting for a follow-up campaign.

Q4 – Customers with Above-Average Sales

SQL: CTE + Subquery referencing the CTE

```
WITH customer_totals AS (
  SELECT
    c.Customer_Name,
    c.Segment,
    c.Region,
    ROUND(SUM(o.Sales), 2) AS total_sales
  FROM orders o
  JOIN customers c ON o.Customer_ID = c.Customer_ID
  GROUP BY c.Customer_Name, c.Segment, c.Region
)
SELECT
  Customer_Name, Segment, Region, total_sales,
  ROUND((SELECT AVG(total_sales) FROM customer_totals), 2) AS avg_sales
FROM customer_totals
WHERE total_sales > (SELECT AVG(total_sales) FROM customer_totals)
ORDER BY total_sales DESC
LIMIT 10;
```

Average customer sales = \$2,896.85 | Above-average customers: 294 / 793 (37.1%)

Customer Name	Segment	Region	Total Sales (\$)	Avg (\$)
Sean Miller	Home Office	South	25,043.05	2,896.85
Tamara Chand	Corporate	West	19,052.22	2,896.85
Raymond Buch	Consumer	East	15,117.34	2,896.85
Tom Ashbrook	Home Office	East	14,595.62	2,896.85
Adrian Barton	Consumer	West	14,473.57	2,896.85
Ken Lonsdale	Consumer	Central	14,175.23	2,896.85
Sanjit Chand	Consumer	West	14,142.33	2,896.85
Hunter Lopez	Consumer	Central	12,873.30	2,896.85
Sanjit Engle	Consumer	East	12,209.44	2,896.85
Christopher Conant	Consumer	South	12,129.07	2,896.85

Insight: 37.1% of customers exceed average — the top quartile drives the bulk of revenue. The Consumer segment dominates the above-average list (7 of top 10), making it the most valuable segment by revenue volume.

Q5 – Highest Order Value per Customer

SQL: CTE chaining (best order per customer) + RANK()

```
WITH order_totals AS (
  -- Total sales per order
  SELECT Order_ID, Customer_ID,
         ROUND(SUM(Sales), 2) AS order_value
  FROM orders
  GROUP BY Order_ID, Customer_ID
),
cust_best AS (
  -- Best (max) order per customer
  SELECT ot.Customer_ID, c.Customer_Name, c.Segment, c.Region,
         MAX(ot.order_value) AS highest_order_value
  FROM order_totals ot
  JOIN customers c ON ot.Customer_ID = c.Customer_ID
  GROUP BY ot.Customer_ID, c.Customer_Name, c.Segment, c.Region
)
SELECT
  Customer_Name, Segment, Region,
  highest_order_value,
  RANK() OVER (ORDER BY highest_order_value DESC) AS value_rank
FROM cust_best
ORDER BY value_rank
LIMIT 10;
```

Rank	Customer Name	Segment	Region	Highest Order (\$)
1	Sean Miller	Home Office	South	23,661.23
2	Tamara Chand	Corporate	West	18,336.74
3	Raymond Buch	Consumer	East	14,052.48
4	Tom Ashbrook	Home Office	East	13,716.46
5	Becky Martin	Consumer	Central	10,539.90
6	Hunter Lopez	Consumer	Central	10,499.97
7	Sanjit Chand	Consumer	West	9,900.19
8	Adrian Barton	Consumer	West	9,892.74
9	Bill Shonely	Corporate	East	9,135.19
10	Sanjit Engle	Consumer	East	8,805.04

Insight: Sean Miller's single largest order (\$23,661) accounts for 94.5% of his total sales — extreme order concentration. Becky Martin (#5) appears only here, not in the top-5 by total sales, indicating she placed one massive order rather than many.

SQL Concepts Summary

Quick reference for all techniques used in this assignment.

Concept	Syntax Pattern	Used In
Subquery (WHERE)	WHERE col = (SELECT MAX(col) FROM ...)	Q1, Q2
Correlated Subquery	WHERE col = (SELECT ... WHERE inner.id = outer.id)	Q2
CTE – WITH clause	WITH cte AS (SELECT ...) SELECT * FROM cte	Q3–Q7, Mini Project
CTE + Subquery	WHERE col > (SELECT AVG(col) FROM same_cte)	Q4, Mini Q4
ROW_NUMBER()	ROW_NUMBER() OVER (PARTITION BY x ORDER BY y)	Q6
RANK()	RANK() OVER (ORDER BY col DESC)	Q5, Q7, Final
DENSE_RANK()	DENSE_RANK() OVER (ORDER BY col DESC)	Q5, Final
JOIN + CTE + Window	WITH ... SELECT ..., RANK() OVER (...) FROM cte JOIN ...	Step 3 Final
PARTITION BY	OVER (PARTITION BY customer_id ORDER BY date)	Q6
HAVING	GROUP BY ... HAVING COUNT(DISTINCT col) = 1	Mini Q3

— End of Week 3 Assignment —